Search Algorithms and Computational Complexity

From Mazes to Minimax and the Tetris Autoplayer

Martin Benning

University College London
ENGF0034 – Design and Professional Skills

3 November 2026



Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Lecture Overview

Learning Objectives

By the end of this lecture, you will be able to:

1. Contrast local rules with systematic search, and navigate mazes using recursive Depth-First Search (DFS) with a visited set.
2. Model two-player zero-sum games as game trees, and analyse combinatorial branching complexity ($\mathcal{O}(b^d)$).
3. Apply the Minimax algorithm to find optimal strategies against rational opponents.
4. Implement mutual recursion, unit testing, and safe state cloning with `deepcopy` in Python (`oxo.py`).
5. Overcome game tree intractability using bounded horizons, heuristics, Alpha-Beta pruning, and target-state search for Assignment 4.

Lecture Roadmap

Single-Agent Search: Mazes and DFS
Adversarial Search and Game Trees
The Minimax Algorithm
Implementation Analysis: oxo.py
Heuristics and the Tetris Autoplayer

Single-Agent Search: Mazes and DFS

The Search Problem: Navigating a Maze

The Goal

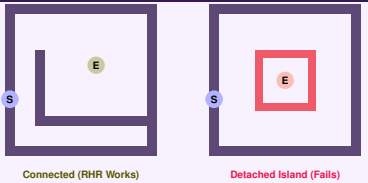
Find a guaranteed path from Start (S) to End (E).

The Right-Hand Rule (Wall Follower)

Keep your right hand against the wall and walk.

- Works in simply connected mazes (like Hampton Court Palace).
- Fails when the goal sits inside an isolated loop (an “island”).
- Without memory, wall followers become trapped in infinite loops.

Connected vs Disconnected Topology



The Takeaway

Memoryless rules fail on general mazes. A general solver must record visited locations.

The Search Problem: Navigating a Maze

The Goal

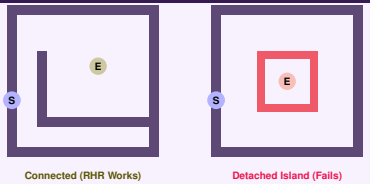
Find a guaranteed path from Start (S) to End (E).

The Right-Hand Rule (Wall Follower)

Keep your right hand against the wall and walk.

- Works in simply connected mazes (like Hampton Court Palace).
- Fails when the goal sits inside an isolated loop (an “island”).
- Without memory, wall followers become trapped in infinite loops.

Connected vs Disconnected Topology



The Takeaway

Memoryless rules fail on general mazes. A general solver must record visited locations.

From Tree Recursion to Graph Search

Trees vs Mazes

- **Week 4 Trees:** Acyclic hierarchy. Every child has one parent, so search cannot loop.
- **Mazes and Graphs:** Contain cycles and multiple paths between the same squares.

Two Requirements for a Solver

1. **Termination:** Avoid infinite loops by storing visited squares in a Python `set` (Week 4).
2. **Completeness:** Explore all reachable paths until the exit is found.

Depth-First Search (DFS)

Follow one path as deep as possible. At a dead end, step back to the last junction and try the next branch.

The Role of the Call Stack

Recursive DFS uses the call stack automatically:

- Stepping forward pushes a stack frame.
- Dead ends pop the frame back to the caller.
- Stack depth is $\mathcal{O}(D)$ for path length D.

From Tree Recursion to Graph Search

Trees vs Mazes

- **Week 4 Trees:** Acyclic hierarchy. Every child has one parent, so search cannot loop.
- **Mazes and Graphs:** Contain cycles and multiple paths between the same squares.

Two Requirements for a Solver

1. **Termination:** Avoid infinite loops by storing visited squares in a Python `set` (Week 4).
2. **Completeness:** Explore all reachable paths until the exit is found.

Depth-First Search (DFS)

Follow one path as deep as possible. At a dead end, step back to the last junction and try the next branch.

The Role of the Call Stack

Recursive DFS uses the call stack automatically:

- Stepping forward pushes a stack frame.
- Dead ends pop the frame back to the caller.
- Stack depth is $\mathcal{O}(\mathrm{D})$ for path length D.

The Recursive Depth-First Search Algorithm

```python
def solve(maze, current, end, visited):
  # Base Case 1: Destination reached
  if current == end:
    return True

  # Base Case 2: Wall or already visited
  if current in visited or maze.is_wall(current):
    return False

  # Mark location as visited (Week 4 set)
  visited.add(current)

  # Recursive Step: Explore each neighbour
  for neighbour in maze.neighbours(current):
    if solve(maze, neighbour, end, visited):
      return True

  # Dead end: return False to backtrack
  return False
```

Algorithm Mechanics

1. **Success:** If at exit, return `True`.
2. **Cycle Prevention:** If current square is in `visited`, stop and return `False`.
3. **Exploration:** Recursively test North, East, South, West.
4. **Backtracking:** If all directions return `False`, this branch is exhausted.

Completeness Guaranteed

Marking visited squares in $\mathcal{O}(1)$ average time and checking every neighbour guarantees finding a path if one exists.

Backtracking: Unwinding the Call Stack

What Happens at a Dead End?

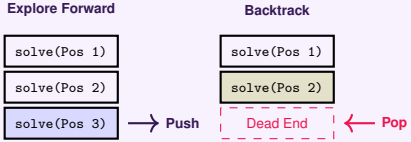
When all neighbours of a square return `False`:

- The current function call finishes and returns `False`.
- Its stack frame is popped from the call stack.
- Execution resumes in the caller, which tries its next unexplored neighbour.

Mazes vs Games: State Restoration

For maze reachability, `visited` stays marked permanently. When comparing move sequences in games, each branch needs a clean board state (via undo or cloning).

Call Stack Progression



Bridge to Games

Game search uses the same depth-first backtracking, with players alternating turns at each level.

Backtracking: Unwinding the Call Stack

What Happens at a Dead End?

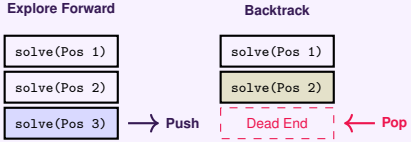
When all neighbours of a square return `False`:

- The current function call finishes and returns `False`.
- Its stack frame is popped from the call stack.
- Execution resumes in the caller, which tries its next unexplored neighbour.

Mazes vs Games: State Restoration

For maze reachability, `visited` stays marked permanently. When comparing move sequences in games, each branch needs a clean board state (via undo or cloning).

Call Stack Progression



Bridge to Games

Game search uses the same depth-first backtracking, with players alternating turns at each level.

Adversarial Search and Game Trees

From Single-Agent to Two-Player Games

Adversarial Environments

In a maze, the environment is passive. In games, we face an opponent actively working against us.

Two-Player Zero-Sum Games

We focus on games with four properties:

- **Two Players:** MAX (maximises score) and MIN (minimises MAX's score).
- **Zero-Sum:** Any gain for MAX is an equal loss for MIN.
- **Deterministic:** No dice or randomness.
- **Perfect Information:** Both players see the entire board (e.g. Tic-Tac-Toe, Chess).

The Game Tree

A game tree models all legal play sequences:

- **Root:** Current board position.
- **Nodes & Edges:** Board states and legal moves.
- **Leaves:** Terminal states (Win, Loss, Draw).

Search Strategy

We explore the game tree with recursive DFS, alternating between MAX and MIN at each depth.

From Single-Agent to Two-Player Games

Adversarial Environments

In a maze, the environment is passive. In games, we face an opponent actively working against us.

Two-Player Zero-Sum Games

We focus on games with four properties:

- **Two Players:** MAX (maximises score) and MIN (minimises MAX's score).
- **Zero-Sum:** Any gain for MAX is an equal loss for MIN.
- **Deterministic:** No dice or randomness.
- **Perfect Information:** Both players see the entire board (e.g. Tic-Tac-Toe, Chess).

The Game Tree

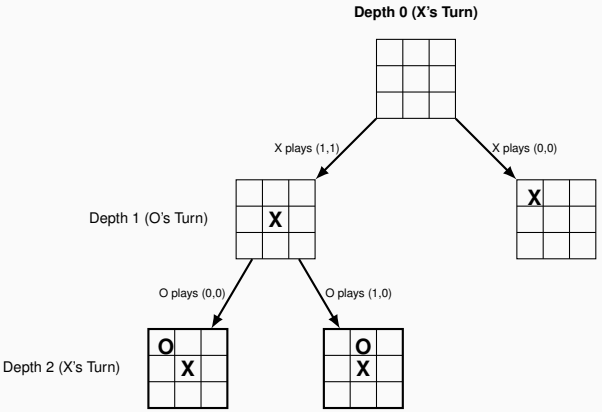
A game tree models all legal play sequences:

- **Root:** Current board position.
- **Nodes & Edges:** Board states and legal moves.
- **Leaves:** Terminal states (Win, Loss, Draw).

Search Strategy

We explore the game tree with recursive DFS, alternating between MAX and MIN at each depth.

A Partial Tic-Tac-Toe Game Tree



Combinatorial Explosion: Tic-Tac-Toe vs Chess

Measuring Complexity

- b: Branching factor (average legal moves per turn).
- d: Maximum game depth in turns (ply).
- Total game tree size: $\mathcal{O}(\mathrm{b}^{\mathrm{d}})$.

Tic-Tac-Toe ($\mathrm{b} \leqslant 9, \mathrm{d} = 9$)

- Naive upper bound: $9! = 362,880$.
- Exact terminal sequences: **255,168**.
- Reachable board states: **5,478** (765 by symmetry).
- Exhaustive search completes in milliseconds.

Chess ($\mathrm{b} \approx 35, \mathrm{d} \approx 80$–$100$ ply)

- Tree size: $35^{80} \approx 10^{120}$ (Shannon's number).
- Observable universe holds $\approx 10^{80}$ atoms.
- Searching this tree exhaustively is impossible.

The Takeaway

Exhaustive search works for Tic-Tac-Toe, but complex games require depth limits and heuristics.

Combinatorial Explosion: Tic-Tac-Toe vs Chess

Measuring Complexity

- b: Branching factor (average legal moves per turn).
- d: Maximum game depth in turns (ply).
- Total game tree size: $\mathcal{O}(\mathrm{b}^{\mathrm{d}})$.

Tic-Tac-Toe ($\mathrm{b} \leqslant 9, \mathrm{d} = 9$)

- Naive upper bound: $9! = 362,880$.
- Exact terminal sequences: **255,168**.
- Reachable board states: **5,478** (765 by symmetry).
- Exhaustive search completes in milliseconds.

Chess ($\mathrm{b} \approx 35$, $\mathrm{d} \approx 80$–100 ply)

- Tree size: $35^{80} \approx \mathbf{10^{120}}$ (Shannon's number).
- Observable universe holds $\approx 10^{80}$ atoms.
- Searching this tree exhaustively is impossible.

The Takeaway

Exhaustive search works for Tic-Tac-Toe, but complex games require depth limits and heuristics.

The Minimax Algorithm

The Minimax Principle

Playing Against a Rational Opponent

We want an algorithm that chooses the best move, assuming our opponent also plays optimally to beat us.

Utilities for MAX

Numerical values assigned to terminal outcomes:

- **Win:** $+1$
- **Draw:** 0
- **Loss:** -1

The Minimax Principle

"Maximise your minimum possible return."

- **MAX** chooses the move with the highest score.
- **MIN** chooses the move that minimises MAX's score.

The Minimax Value $V(s)$

The score that MAX can guarantee from state s against an optimal opponent.

The Minimax Principle

Playing Against a Rational Opponent

We want an algorithm that chooses the best move, assuming our opponent also plays optimally to beat us.

Utilities for MAX

Numerical values assigned to terminal outcomes:

- **Win:** $+1$
- **Draw:** 0
- **Loss:** -1

The Minimax Principle

"Maximise your minimum possible return."

- **MAX** chooses the move with the highest score.
- **MIN** chooses the move that minimises MAX's score.

The Minimax Value $V(s)$

The score that MAX can guarantee from state s against an optimal opponent.

The Minimax Recurrence Relation

Recursive Definition of $V(s)$

The Minimax value is defined from terminal leaves back up to the root,

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal,} \\ \max\limits_{s' \in \text{Children}(s)} V(s') & \text{if Player}(s) = \text{MAX,} \\ \min\limits_{s' \in \text{Children}(s)} V(s') & \text{if Player}(s) = \text{MIN.} \end{cases}$$

How It Works

1. **Base Case:** Score terminal board states directly $(+1, 0, -1)$.
2. **Back up Values:** Propagate scores upward through the tree.
3. **Decision:** MAX chooses the branch leading to the highest value.

Depth-First Traversal

Minimax is implemented using recursive DFS on the game tree.

The Minimax Recurrence Relation

Recursive Definition of $V(s)$

The Minimax value is defined from terminal leaves back up to the root,

$$V(s) = \begin{cases} \text{Utility}(s) & \text{if } s \text{ is terminal,} \\ \max\limits_{s' \in \text{Children}(s)} V(s') & \text{if Player}(s) = \text{MAX,} \\ \min\limits_{s' \in \text{Children}(s)} V(s') & \text{if Player}(s) = \text{MIN.} \end{cases}$$

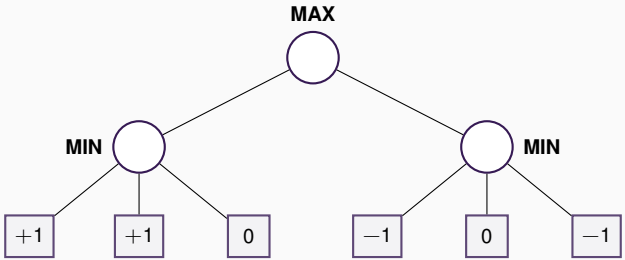
How It Works

1. **Base Case:** Score terminal board states directly $(+1, 0, -1)$.
2. **Back up Values:** Propagate scores upward through the tree.
3. **Decision:** MAX chooses the branch leading to the highest value.

Depth-First Traversal

Minimax is implemented using recursive DFS on the game tree.

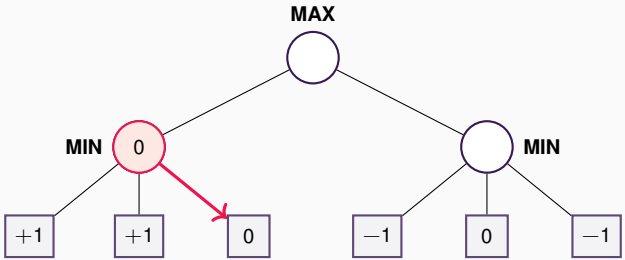
Visualising Minimax Value Propagation



Optimal Play Outcome (and Tic-Tac-Toe)

MIN chooses $\min(+1, +1, 0) = 0$ on the left and $\min(-1, 0, -1) = -1$ on the right; MAX chooses $\max(0, -1) = 0$. On an empty Tic-Tac-Toe board, $V(\text{root}) = 0$: optimal play from both sides always ends in a draw.

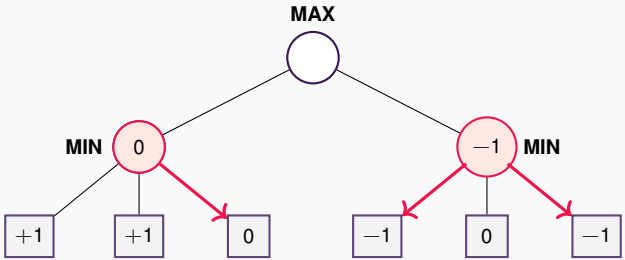
Visualising Minimax Value Propagation



Optimal Play Outcome (and Tic-Tac-Toe)

MIN chooses $\min(+1, +1, 0) = 0$ on the left and $\min(-1, 0, -1) = -1$ on the right; MAX chooses $\max(0, -1) = 0$. On an empty Tic-Tac-Toe board, $V(\text{root}) = 0$: optimal play from both sides always ends in a draw.

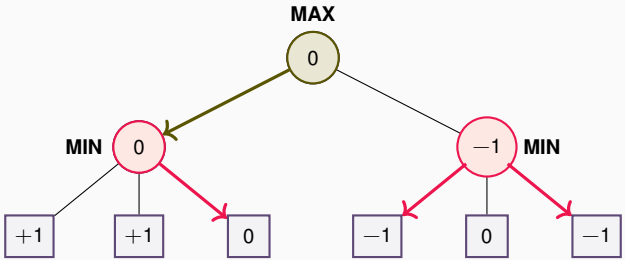
Visualising Minimax Value Propagation



Optimal Play Outcome (and Tic-Tac-Toe)

MIN chooses $\min(+1, +1, 0) = 0$ on the left and $\min(-1, 0, -1) = -1$ on the right; MAX chooses $\max(0, -1) = 0$. On an empty Tic-Tac-Toe board, $V(\text{root}) = 0$: optimal play from both sides always ends in a draw.

Visualising Minimax Value Propagation



Optimal Play Outcome (and Tic-Tac-Toe)

MIN chooses $\min(+1, +1, 0) = 0$ on the left and $\min(-1, 0, -1) = -1$ on the right; MAX chooses $\max(0, -1) = 0$. On an empty Tic-Tac-Toe board, $V(\text{root}) = 0$: optimal play from both sides always ends in a draw.

Implementation Analysis: oxo.py

Board Representation and Rules (`oxo.py`)

Separation of Concerns

The code separates the game mechanics from the decision-making AI:

- `Board` class: maintains grid state and enforces rules.
- `Oxo` class: implements the Minimax search strategy.

Inside the `Board` Class

- `self.pieces`: 3×3 nested list of characters (`'X'`, `'O'`, `'.'`).
- `self.count`: tracks played pieces; `unfinished()` checks `count < 9` in $\mathcal{O}(1)$ time.

Win Detection: `wins(player)`

Checks rows, columns, and two diagonals in a single loop through the 9 board cells. Because board dimensions are fixed at 3×3, `wins()` runs in constant $\mathcal{O}(1)$ time.

Efficiency Matters

During the search, `wins()` is called thousands of times. An efficient $\mathcal{O}(1)$ check keeps the recursion fast.

Exploring Hypothetical Moves: State Cloning

The Need for a Sandbox

During search, the AI simulates many possible future moves. We must never overwrite the active game board.

The Cloning Solution

Create an independent copy before making a trial move:

```
1 from copy import deepcopy
2
3 class Oxo:
4   def clone(self):
5     return deepcopy(self)
```

Why Deepcopy?

Recalling Week 2: `Board.pieces` is a nested list. A shallow copy (`list.copy()`) copies only the outer list, leaving row lists shared. `deepcopy` gives full isolation.

Assignment 4 Link

In Assignment 4 (Tetris), test candidate placements on an isolated clone via `sandbox = board.clone()`.

Exploring Hypothetical Moves: State Cloning

The Need for a Sandbox

During search, the AI simulates many possible future moves. We must never overwrite the active game board.

The Cloning Solution

Create an independent copy before making a trial move:

```
1 from copy import deepcopy
2
3 class Oxo:
4   def clone(self):
5     return deepcopy(self)
```

Why Deepcopy?

Recalling Week 2: `Board.pieces` is a nested list. A shallow copy (`list.copy()`) copies only the outer list, leaving row lists shared. `deepcopy` gives full isolation.

Assignment 4 Link

In Assignment 4 (Tetris), test candidate placements on an isolated clone via `sandbox = board.clone()`.

Mutual Recursion and Symbolic Preferences

Symbolic Constants in `oxo.py`

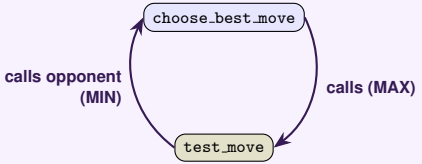
Instead of numbers $+1, 0, -1$, the implementation defines:

- `WIN_IMMEDIATE = 0` (win on this move)
- `WIN = 1` (guaranteed forced win)
- `LOSE = 2` (forced loss)
- `DRAW = 3` (forced draw)

The preference ordering satisfies

$$\texttt{WIN_IMMEDIATE} \succ \texttt{WIN} \succ \texttt{DRAW} \succ \texttt{LOSE}.$$

Two Mutually Recursive Functions



Dynamic Tree Generation

These two methods call each other down to terminal leaves, generating the game tree on the fly.

The Recursive Evaluator: `Oxo.test_move`

```
def test_move(self, pos, player):
  # 1. Clone state and make the move
  game = self.clone()
  game.board.play(pos, player)

  # 2. Base Case: Check for immediate win
  if game.board.wins(player):
    return WIN_IMMEDIATE

  # 3. Recursive Step: Opponent's reply
  result = game.choose_best_move(
    other_player(player))

  # 4. Zero-Sum Inversion:
  if result == WIN:
    return LOSE
  elif result == LOSE:
    return WIN
  return DRAW
```

The Minimax Inversion

In a zero-sum game, our move's value is the inverse of the opponent's best reply:

- Opponent forces `WIN` $\Rightarrow$ our `LOSE`.
- Opponent forced to `LOSE` $\Rightarrow$ our `WIN`.
- Otherwise $\Rightarrow$ `DRAW`.

MIN Logic

Inverting the opponent's outcome implements the MIN step of Minimax.

The Search Driver: `Oxo.choose_best_move`

```python
def choose_best_move(self, player):
  if not self.board.unfinished():
    return DRAW  # Full board

  winning = drawing = losing = None
  for y in range(3):
    for x in range(3):
      pos = (x, y)
      if not self.board.empty(pos):
        continue
      res = self.test_move(pos, player)
      if res == WIN_IMMEDIATE:
        self.board.play(pos, player)
        return WIN  # Immediate win
      if res == WIN: winning = pos
      elif res == DRAW: drawing = pos
      elif res == LOSE: losing = pos

  move = winning or drawing or losing
  self.board.play(move, player)
  return WIN if winning else (DRAW if drawing else LOSE)
```

MAX Logic

1. Tests open squares with `test_move`.
2. **Short-Circuit:** Returns immediately on `WIN_IMMEDIATE`.
3. Otherwise selects best available outcome: Win $\succ$ Draw $\succ$ Lose.

Ties

If several moves share the best outcome (e.g. multiple draws), any valid move can be chosen.

Testing with TDD and Minimax Complexity

Unit Testing with `assert`

```python
# test_oxo.py: Immediate win check
game = Oxo()
game.board.set("XO.",
               "XO.",
               "...")
# X must play at (0, 2) to win
assert(game.choose_best_move("X") == WIN)

# Empty board must evaluate to a draw
game = Oxo()
assert(game.choose_best_move("X") == DRAW)
```

Complexity of Pure Minimax

- **Time Complexity:** $\mathcal{O}(\mathrm{b}^{\mathrm{d}})$. Visits every reachable node in the game tree.
- **Space Complexity:** $\mathcal{O}(\mathrm{d})$. Stores active stack frames along the current branch.

The Intractability Limit

Searching to terminal nodes is feasible for Tic-Tac-Toe ($5,478$ states), but impossible for Chess (10^{120}) or Tetris (5^{20}).

Heuristics and the Tetris Autoplayer

Handling Intractability: Bounded Horizons

The Problem

For complex games, we cannot search to the end of the game tree.

Two Complementary Solutions

1. **Bounded Search Horizon:** Stop the search at a fixed depth limit h.
2. **Heuristic Evaluation Function** $E(s)$**:** Estimate the strategic quality of non-terminal board positions at depth h.

Chess Example: Material Heuristic

Count piece values (Pawn $= 1$, Knight/Bishop $= 3$, Rook $= 5$, Queen $= 9$):

$$E(s) = \sum \text{our pieces} - \sum \text{their pieces}.$$

Trade-off and the Horizon Effect

We trade guaranteed optimality for speed. A threat pushed just past depth h stays invisible until later (**Horizon Effect**).

Pruning the Search Space: Alpha-Beta Intuition

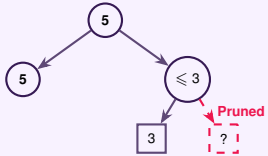
Do We Need to Search Everything?

Can we skip branches that cannot affect the final decision?

Intuitive Example

1. MAX evaluates Move A and finds it guarantees a score of at least 5.
2. MAX begins evaluating Move B.
3. MAX discovers that MIN can respond to B by forcing a score of 3.
4. MAX stops evaluating Move B immediately: since $3 < 5$, Move B will never be chosen.

Alpha-Beta Pruning Impact



Performance Gain

With good move ordering, Alpha-Beta reduces the effective branching factor from $\mathfrak{b}$ to $\sqrt{\mathfrak{b}}$, doubling reachable search depth.

Assignment 4: Why Exhaustive Action Search Fails

Approach 1: Searching Action Sequences

Try every sequence of moves (`Left`, `Right`, `Rotate`, `Drop`, `Skip`):

- At each step, roughly 5 choices over $\approx$ 20 rows.
- Paths per falling block grow as

$$5^{20} \approx \mathbf{9.5} \times \mathbf{10^{13}}.$$

- Exceeds autograder memory (256 MB) and CPU limits.

Approach 2: Target-State Search

Focus on where the piece can **land** (column + rotation):

- **1-piece lookahead:**
  10 cols $\times$ 4 rotations $\approx$ $\mathbf{40}$ states.
- **2-piece lookahead** (with `board.next`):
  $40 \times 40 \approx \mathbf{1{,}600}$ states.

Feasible and Fast

Evaluating 40 (or 1,600) states finishes in milliseconds. Stream the winning actions via `return` or `yield` (Week 4).

Assignment 4: Why Exhaustive Action Search Fails

Approach 1: Searching Action Sequences

Try every sequence of moves (`Left`, `Right`, `Rotate`, `Drop`, `Skip`):

- At each step, roughly 5 choices over $\approx$ 20 rows.
- Paths per falling block grow as

$$5^{20} \approx \mathbf{9.5} \times \mathbf{10^{13}}.$$

- Exceeds autograder memory (256 MB) and CPU limits.

Approach 2: Target-State Search

Focus on where the piece can **land** (column + rotation):

- **1-piece lookahead:**
  10 cols $\times$ 4 rotations $\approx$ $\mathbf{40}$ states.
- **2-piece lookahead** (with `board.next`):
  40 $\times$ 40 $\approx$ $\mathbf{1{,}600}$ states.

Feasible and Fast

Evaluating 40 (or 1,600) states finishes in milliseconds. Stream the winning actions via `return` or `yield` (Week 4).

Scoring the Board: Designing Tetris Heuristics

The Simulation Workflow

For each candidate (rotation, column) target:

1. Clone board: `sandbox = board.clone()`.
2. Rotate and move horizontally to target column.
3. Drop immediately with `Direction.Drop`.
4. Score `sandbox.cells` (origin `(0, 0)` is top-left).

Pick the placement with the highest heuristic score (start without Bombs or Discards, then add them once stable).

Linear Heuristic Evaluation

Combine board features with tuned weights,

$$\mathrm{E}(s) = w_1 \mathrm{L} - w_2 \mathrm{H} - w_3 \mathrm{K} - w_4 \mathrm{B}$$

- L: Lines cleared (1 row $=$ 25, 4 rows $=$ 1,600).
- H: Aggregate column height (penalise).
- K: Buried holes below blocks (penalise heavily).
- B: Surface bumpiness between adjacent columns.

Lecture Summary and Key Takeaways

Single-Agent & Adversarial Search

- **DFS and Cycles:** Marking visited squares prevents infinite loops; the call stack handles backtracking.
- **Game Trees:** Model turns in zero-sum games with complexity $\mathcal{O}(b^d)$.
- **Minimax:** Maximises the guaranteed score against a rational opponent.

Engineering Lessons for Assignment 4

- **State Isolation:** Always clone mutable state (`sandbox = board.clone()`) before testing moves.
- **Complexity Reduction:** Replace 5^{20} action searches with ≈ 40 landing target states.
- **Heuristics:** Penalise buried holes and stack height while rewarding multi-row clears.

Best of luck building your Tetris autoplayers.